JAVA PROGRAMMING INTERNSHIP PROJECT

EMPLOYEE PAYROLL MANAGEMENT SYSTEM

Using Java, JDBC and MySQL

Project Type: Console-based Java application

Main Features: Add employees, view records, update salary, generate payslip, delete employee

Prepared for: Java Programming Internship — Task 4

Organization: Syntecxhub

Note: Replace the database password in the configuration before running. This document contains source code and setup instructions; run the program locally to capture your actual output screenshots.

1. Project Introduction

The Employee Payroll Management System is a Java console application designed to maintain employee details and manage monthly salary records. It uses JDBC (Java Database Connectivity) to communicate with a MySQL database. The system supports adding employees, viewing employee records, updating salary details, generating a payslip, and deleting an employee record.

2. Objectives

• Maintain employee information in a structured database.

• Calculate gross salary, deductions, and net salary using a simple, transparent formula.

• Provide CRUD operations for employee records.

• Use JDBC PreparedStatement for parameterized database operations.

• Handle invalid input and database errors gracefully.

3. Tools and Technologies

Technology	Purpose
Java	Application logic and console interface
JDBC	Connects Java to MySQL
MySQL	Stores employee and payroll data
Maven	Manages the MySQL JDBC driver dependency

4. Payroll Calculation

For this educational project, the sample payroll formula is: Gross Salary = Basic Salary + Allowances; Total Deductions = Provident Fund (PF) + Tax; Net Salary = Gross Salary – Total Deductions. Allowance, PF and tax percentages are configurable constants in the code and are simplified examples, not legal or tax advice.

```
ALLOWANCE = 20% of basic salary
PF = 12% of basic salary
TAX = 5% of gross salary
Gross Salary = Basic Salary + Allowance
Net Salary = Gross Salary - PF - TAX
```

5. Database Setup

Create a database named employee_payroll and an employees table. The program also stores salary components in each employee record.

```
CREATE DATABASE IF NOT EXISTS employee_payroll;
USE employee_payroll;

CREATE TABLE IF NOT EXISTS employees (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    department VARCHAR(100) NOT NULL,
    basic_salary DECIMAL(12,2) NOT NULL CHECK (basic_salary >= 0),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

6. Project Structure

```
EmployeePayrollManagement/
  pom.xml
```

```
database/schema.sql
src/main/java/com/syntecxhub/payroll/
  DatabaseConnection.java
  Employee.java
  EmployeeDAO.java
  Main.java
```

7. Source Code

7.1 pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.syntecxhub</groupId>
  <artifactId>employee-payroll-management</artifactId>
  <version>1.0.0</version>
  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencies>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>9.4.0</version>
    </dependency>
  </dependencies>
</project>
```

7.2 DatabaseConnection.java

```
package com.syntecxhub.payroll;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public final class DatabaseConnection {
    private static final String URL =
        System.getenv().getOrDefault("PAYROLL_DB_URL",
        "jdbc:mysql://localhost:3306/employee_payroll");
    private static final String USER =
        System.getenv().getOrDefault("PAYROLL_DB_USER", "root");
    private static final String PASSWORD =
        System.getenv().getOrDefault("PAYROLL_DB_PASSWORD", "change_me");

    private DatabaseConnection() {}

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

7.3 Employee.java

```
package com.syntecxhub.payroll;

import java.math.BigDecimal;

public class Employee {
    private final int id;
    private final String name, email, department;
    private final BigDecimal basicSalary;

    public Employee(int id, String name, String email,
                    String department, BigDecimal basicSalary) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.department = department;
        this.basicSalary = basicSalary;
    }

    public Employee(String name, String email, String department,
                    BigDecimal basicSalary) {
        this(0, name, email, department, basicSalary);
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getDepartment() { return department; }
    public BigDecimal getBasicSalary() { return basicSalary; }
}
```

7.4 EmployeeDAO.java

```
package com.syntecxhub.payroll;

import java.sql.*;
import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;

public class EmployeeDAO {
    public boolean add(Employee e) throws SQLException {
        String sql = "INSERT INTO employees(name,email,department,basic_salary) VALUES(?,?,?,?)";
        try (Connection c = DatabaseConnection.getConnection();
             PreparedStatement p = c.prepareStatement(sql)) {
            p.setString(1, e.getName());
            p.setString(2, e.getEmail());
            p.setString(3, e.getDepartment());
            p.setBigDecimal(4, e.getBasicSalary());
            return p.executeUpdate() == 1;
        }
    }

    public List<Employee> all() throws SQLException {
        List<Employee> list = new ArrayList<>();
        String sql = "SELECT id,name,email,department,basic_salary FROM employees ORDER BY id";
        try (Connection c = DatabaseConnection.getConnection();
             PreparedStatement p = c.prepareStatement(sql);
             ResultSet r = p.executeQuery()) {
            while (r.next()) {
                list.add(new Employee(r.getInt("id"), r.getString("name"),
                    r.getString("email"), r.getString("department"),
                    r.getBigDecimal("basic_salary")));
            }
        }
        return list;
    }

    public boolean updateSalary(int id, BigDecimal salary) throws SQLException {
        String sql = "UPDATE employees SET basic_salary=? WHERE id=?";
        try (Connection c = DatabaseConnection.getConnection();
             PreparedStatement p = c.prepareStatement(sql)) {
            p.setBigDecimal(1, salary);
            p.setInt(2, id);
            return p.executeUpdate() == 1;
        }
    }

    public boolean delete(int id) throws SQLException {
        String sql = "DELETE FROM employees WHERE id=?";
        try (Connection c = DatabaseConnection.getConnection();
             PreparedStatement p = c.prepareStatement(sql)) {
            p.setInt(1, id);
            return p.executeUpdate() == 1;
        }
    }

    public Employee findById(int id) throws SQLException {
        String sql = "SELECT id,name,email,department,basic_salary FROM employees WHERE id=?";
        try (Connection c = DatabaseConnection.getConnection();
             PreparedStatement p = c.prepareStatement(sql)) {
            p.setInt(1, id);
            try (ResultSet r = p.executeQuery()) {
                if (!r.next()) return null;
                return new Employee(r.getInt("id"), r.getString("name"),
                    r.getString("email"), r.getString("department"),
                    r.getBigDecimal("basic_salary"));
            }
        }
    }
}
```

7.5 Main.java

```
package com.syntecxhub.payroll;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.sql.SQLException;
import java.util.List;
import java.util.Scanner;

public class Main {
```

```
private static final Scanner IN = new Scanner(System.in);
private static final EmployeeDAO DAO = new EmployeeDAO();
private static final BigDecimal ALLOWANCE_RATE = new BigDecimal("0.20");
private static final BigDecimal PF_RATE = new BigDecimal("0.12");
private static final BigDecimal TAX_RATE = new BigDecimal("0.05");

public static void main(String[] args) {
    boolean running = true;
    System.out.println("EMPLOYEE PAYROLL MANAGEMENT SYSTEM");
    while (running) {
        System.out.println("\n1. Add employee\n2. View employees"
            + "\n3. Update salary\n4. Generate payslip"
            + "\n5. Delete employee\n6. Exit");
        int choice = readInt("Choose: ");
        try {
            switch (choice) {
                case 1 -> addEmployee();
                case 2 -> viewEmployees();
                case 3 -> updateSalary();
                case 4 -> payslip();
                case 5 -> deleteEmployee();
                case 6 -> running = false;
                default -> System.out.println("Invalid menu choice.");
            }
        } catch (SQLException ex) {
            System.out.println("Database error: " + ex.getMessage());
            System.out.println("Check MySQL service and connection settings.");
        }
    }
    IN.close();
    System.out.println("Goodbye!");
}

private static void addEmployee() throws SQLException {
    String name = required("Name: ");
    String email = required("Email: ");
    String dept = required("Department: ");
    BigDecimal salary = readMoney("Basic monthly salary: ");
    if (salary.signum() < 0) {
        System.out.println("Salary cannot be negative."); return;
    }
    DAO.add(new Employee(name, email, dept, salary));
    System.out.println("Employee added.");
}

private static void viewEmployees() throws SQLException {
    List<Employee> list = DAO.all();
    if (list.isEmpty()) { System.out.println("No employee records."); return; }
    System.out.printf("%-5s %-20s %-28s %-18s %12s%n",
        "ID", "NAME", "EMAIL", "DEPARTMENT", "BASIC");
    for (Employee e : list)
        System.out.printf("%-5d %-20s %-28s %-18s %12s%n",
            e.getId(), e.getName(), e.getEmail(), e.getDepartment(),
            money(e.getBasicSalary()));
}

private static void updateSalary() throws SQLException {
    int id = readInt("Employee ID: ");
    BigDecimal salary = readMoney("New basic salary: ");
    if (salary.signum() < 0) { System.out.println("Salary cannot be negative."); return; }
    System.out.println(DAO.updateSalary(id, salary)
        ? "Salary updated." : "Employee ID not found.");
}

private static void payslip() throws SQLException {
    int id = readInt("Employee ID: ");
    Employee e = DAO.findById(id);
    if (e == null) { System.out.println("Employee ID not found."); return; }

    BigDecimal basic = e.getBasicSalary();
    BigDecimal allowance = basic.multiply(ALLOWANCE_RATE);
    BigDecimal gross = basic.add(allowance);
    BigDecimal pf = basic.multiply(PF_RATE);
    BigDecimal tax = gross.multiply(TAX_RATE);
    BigDecimal deductions = pf.add(tax);
    BigDecimal net = gross.subtract(deductions);

    System.out.println("\n========== MONTHLY PAYSLIP ==========");
    System.out.println("Employee ID : " + e.getId());
    System.out.println("Name        : " + e.getName());
    System.out.println("Department  : " + e.getDepartment());
    System.out.println("-------------------------------------");
```

```
        System.out.println("Basic salary: " + money(basic));
        System.out.println("Allowance   : " + money(allowance));
        System.out.println("Gross salary: " + money(gross));
        System.out.println("PF deduction: " + money(pf));
        System.out.println("Tax example : " + money(tax));
        System.out.println("Deductions  : " + money(deductions));
        System.out.println("NET SALARY  : " + money(net));
        System.out.println("====================================");
    }

    private static void deleteEmployee() throws SQLException {
        int id = readInt("Employee ID to delete: ");
        System.out.print("Type YES to confirm deletion: ");
        if (!"YES".equalsIgnoreCase(IN.nextLine().trim())) {
            System.out.println("Deletion cancelled."); return;
        }
        System.out.println(DAO.delete(id) ? "Employee deleted." : "Employee ID not found.");
    }

    private static String required(String prompt) {
        while (true) {
            System.out.print(prompt);
            String s = IN.nextLine().trim();
            if (!s.isEmpty()) return s;
            System.out.println("This field is required.");
        }
    }

    private static int readInt(String prompt) {
        while (true) {
            System.out.print(prompt);
            try { return Integer.parseInt(IN.nextLine().trim()); }
            catch (NumberFormatException ex) { System.out.println("Enter a whole number."); }
        }
    }

    private static BigDecimal readMoney(String prompt) {
        while (true) {
            System.out.print(prompt);
            try { return new BigDecimal(IN.nextLine().trim()).setScale(2, RoundingMode.HALF_UP); }
            catch (NumberFormatException ex) { System.out.println("Enter a valid amount, e.g. 25000.00"); }
        }
    }

    private static String money(BigDecimal value) {
        return "Rs. " + value.setScale(2, RoundingMode.HALF_UP).toPlainString();
    }
}
```

8. How to Run the Project

• Install JDK 17 or later, MySQL Server, and Maven.

• Open MySQL Workbench or a MySQL command line and execute database/schema.sql.

• Set environment variables PAYROLL_DB_USER and PAYROLL_DB_PASSWORD, or edit the default values in DatabaseConnection.java. Do not commit real passwords to GitHub.

• Open a terminal in the project folder and run: mvn compile

• Run the Main class from your IDE, or use Maven exec configuration in your IDE. The MySQL Connector/J dependency is declared in pom.xml.

• Use the menu to add an employee, view employees, update salary, generate a payslip, or delete an employee.

9. Sample Output (Illustrative)

The following is an example of the kind of output the application prints. It is illustrative only; your actual output will depend on the data entered and a successful MySQL connection.

```
EMPLOYEE PAYROLL MANAGEMENT SYSTEM

1. Add employee
2. View employees
3. Update salary
4. Generate payslip
5. Delete employee
6. Exit
Choose: 1
Name: Riya Sharma
Email: riya@example.com
Department: IT
Basic monthly salary: 25000
Employee added.

Choose: 4
Employee ID: 1

========== MONTHLY PAYSLIP ==========
Employee ID : 1
Name        : Riya Sharma
Department  : IT
Basic salary: Rs. 25000.00
Allowance   : Rs. 5000.00
Gross salary: Rs. 30000.00
PF deduction: Rs. 3000.00
Tax example : Rs. 1500.00
Deductions  : Rs. 4500.00
NET SALARY  : Rs. 25500.00
=====================================
```

10. Security and Error Handling

• PreparedStatement is used for database queries to reduce SQL injection risk.

• Try-with-resources closes database connections and statements automatically.

• Email is unique in the database to avoid duplicate employee accounts.

• The program checks numeric input and rejects negative salary values.

• Database credentials should be kept out of source control. Use environment variables or a local configuration file excluded from Git.

11. Limitations and Future Enhancements

This is a learning project. The payroll percentages are illustrative and do not represent official tax or statutory calculations. Future versions could add a graphical user interface, role-based login, month-wise payroll history, PDF

payslip export, input validation for email and phone, audit logging, and automated tests.

12. Conclusion

The Employee Payroll Management System demonstrates core Java programming, JDBC connectivity, MySQL database operations, CRUD functionality, and basic payroll calculations. It provides a foundation that can be extended into a more complete payroll application.

13. Internship Submission Checklist

■ Run the SQL script and verify that the database and table are created.

■ Compile and test each menu option on your own computer.

■ Capture genuine screenshots of your program running successfully.

■ Upload the complete source code to a GitHub repository named Syntecxhub_Employee_Payroll_Management.

■ Submit the project using the official internship submission form.

Important: Review the code, replace the database credentials, and run the application before submitting it. The PDF itself is project documentation; it does not prove the application has been executed.